

MICROSOFT BUILD · DEVELOPER WORKSHOP

Microsoft Agent Framework

Building Production-Ready AI Agents

A hands-on workshop — every concept below ships as a runnable Python example.

Agents

Tools

Memory

Workflows

MCP

Middleware

Production

Satendra Kumar

agent-framework 1.13 · Python 3.13 · uv

Eight modules

01

Why a framework

The gap between a completion and an agent

02

Core concepts

Clients, agents, sessions, the run lifecycle

03

Tools

Typed functions, the call loop, approvals

04

Memory & context

Sessions, providers, persistence, compaction

05

MCP

Standardised tool servers

06

Workflows

Executors, edges, execution, orchestration

07

Middleware

Guardrails and cross-cutting concerns

08

A2A Protocol

Agent to agent talks

Why a framework

What actually breaks when you wire an agent loop by hand

the gap

hand-rolled cost

lineage

packages

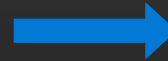
01

A completion is not an agent

CHAT COMPLETION

One shot. Stateless.

prompt in → tokens out. Everything it knows, you put in the prompt. Everything it does, you do afterwards.



AGENT

Loops. Acts. Remembers.

Decides to call a tool, reads the result, decides again — until the goal is met or a budget runs out.

The loop is ten lines. Everything around the loop is the hard part.

state across turns

tool schemas

retries & timeouts

approvals

context limits

traces

cost caps

streaming

What you end up writing yourself

- **Message plumbing**

Roles, tool-call IDs, multi-part content, provider drift.

- **Schema generation**

Python signatures → JSON Schema, kept in sync by hand.

- **The tool loop**

Parse, dispatch, serialise results, guard against loops.

- **Conversation state**

Store, load, trim — and decide what 'trim' means.

- **Provider swaps**

Each SDK disagrees about streaming, tools, and errors.

- **Human approval**

Pausing mid-loop and resuming is genuinely hard.

- **Telemetry**

Spans per turn, per tool, per token — retrofitted, always.

- **Multi-agent**

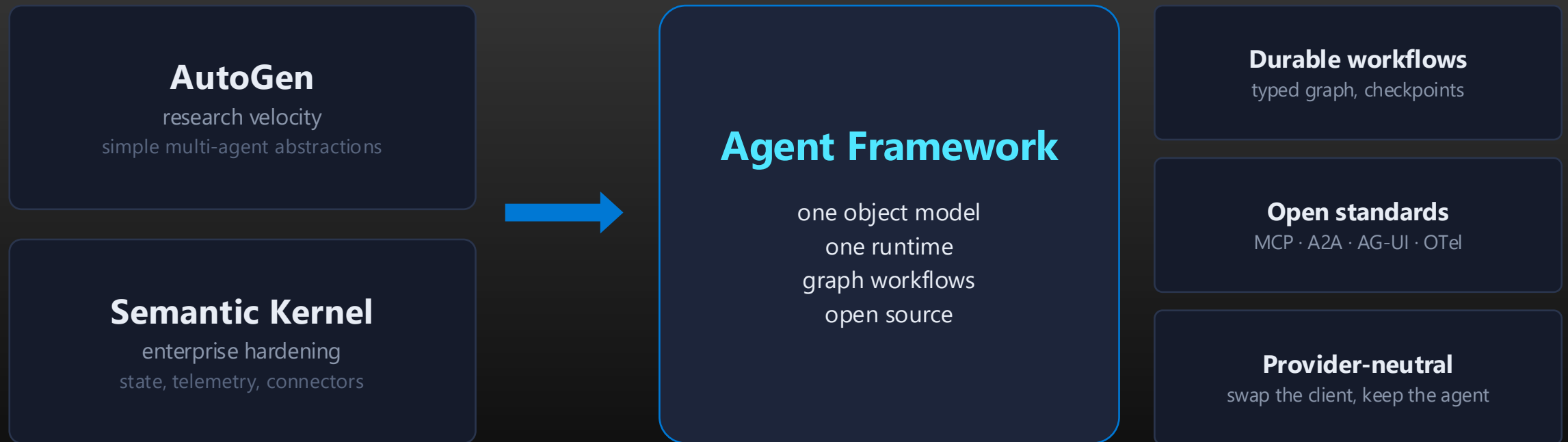
Once there are two agents, you have written an orchestrator.

THE ACTUAL ARGUMENT

None of these are hard problems. They are all boring problems — and you will solve each one slightly differently in every service you own.

Microsoft Agent Framework

One SDK and runtime for agents and multi-agent workflows — Python and .NET.



How the packages are laid out

agent_framework

Agent · tool · AgentSession · ContextProvider · WorkflowBuilder · Executor · middleware · compaction

Model providers

openai · azure
foundry · anthropic
gemini · bedrock
mistral · ollama
claude · copilotstudio

Memory & stores

redis · mem0
azure_cosmos · azure_ai_search

Surfaces & hosting

devui · ag_ui
a2a · azurefunctions
durabletask · chatkit
foundry_hosting

Core concepts

Five primitives, and the lifecycle that connects them

chat client

agent

session

context provider

lifecycle

02

Five primitives

1

Chat client

`OpenAIChatCompletionClient`

Talks to a model.
Swappable.

2

Agent

`Agent(...)`

Instructions + tools +
client. The unit of
behaviour.

3

Tool

`@tool`

A typed Python function
the model may call.

4

Session

`agent.create_session()`

Carries conversation state
across runs.

5

Context provider

`ContextProvider`

Injects and captures
context per run.

COMPOSITION, NOT INHERITANCE

You never subclass Agent. You compose it: hand it a different client, another tool, one more context provider, a middleware. That is the whole extensibility story.

Chat clients: the model is a parameter

the only provider-specific line

```
from agent_framework import Agent
from agent_framework_openai import OpenAIChatCompletionClient

agent = Agent(
    client=OpenAIChatCompletionClient(), # swap this line only
    name="HelloAgent",
    instructions="You are a friendly assistant. Be brief.",
)
```

SAME AGENT, OTHER BACKENDS

AzureOpenAIChatClient	Azure OpenAI
FoundryChatClient	Azure AI Foundry
AnthropicChatClient	Claude
OllamaChatClient	local Ollama

THE ONE DISTINCTION THAT BITES PEOPLE

OpenAIChatCompletionClient → Chat Completions API.

OpenAIChatClient → the newer Responses API. Make sure your server like vLLM and Ollama support response API

Your first agent

```
import asyncio
from agent_framework import Agent
from agent_framework_openai import OpenAIChatCompletionClient
from dotenv import load_dotenv

load_dotenv()

async def main() -> None:
    agent = Agent(
        client=OpenAIChatCompletionClient(),
        name="HelloAgent",
        instructions="Keep your answers brief.",
    )
    result = await agent.run("What is the capital of France?")
    print(result)

asyncio.run(main())
```

- **Async everywhere**

agent.run() is a coroutine. No sync shim.

- **Config from env**

load_dotenv() feeds base URL, key and model.

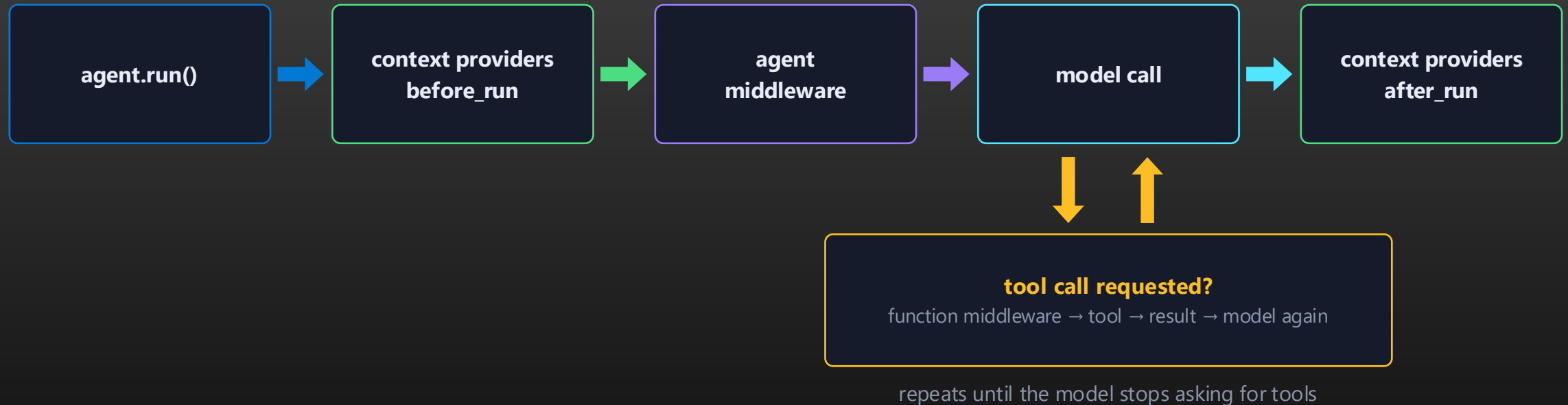
- **Instructions ≠ prompt**

System-level, applied to every run.

- **Result is rich**

result.text, .messages, .value — not just a string.

The run lifecycle



READ THE DIAGRAM AS AN ONION

before_run injects context → middleware wraps the call → the model may loop through tools → after_run captures what happened. Every extension point you will meet today is one of those four rings.

Streaming, options and structured results

```
# non-streaming – one await, full response
result = await agent.run("What is the capital of France?")

# per-call options ride alongside the prompt
result = await agent.run(prompt, options={
    "temperature": 0.3,
    "max_tokens": 150,
})

# streaming – same method, stream=True
async for chunk in agent.run(prompt, stream=True):
    if chunk.text:
        print(chunk.text, end="", flush=True)
```

- **One method, two shapes**
stream=True returns an async iterator.
- **Guard chunk.text**
Chunks may carry tool calls, not text.
- **options is per call**
default_options on Agent sets the baseline.
- **Cap tokens deliberately**
The workshop model has an 8k window.

Demo — hello agent, streaming, options

▶ DEMO

terminal

```
# start the model server first (separate terminal)
./vllm-run.sh

# then:
uv run python examples/01_hello_agent.py
```

WHAT TO POINT AT

- ▶ The first response is blocking — the whole answer lands at once.
- ▶ The second passes temperature and max_tokens per call.
- ▶ The third streams: tokens appear as they are produced.
- ▶ Edit instructions live and re-run — show the behaviour change.

Tools

Giving the model a typed, governed way to touch your systems

@tool

schema

the loop

approvals

03

Anatomy of a tool

```
@tool(approval_mode="never_require")
def get_weather(
    location: Annotated[str, Field(description="City to look up.")],
) -> str:
    """Get the weather for a given location."""
    return f"{location}: sunny, high of 24C"
```

WHAT THE MODEL ACTUALLY RECEIVES

function name	→ <code>get_weather</code>	the callable identifier
docstring	→ <code>"Get the weather..."</code>	when to call it
type hints	→ <code>location: str</code>	types → JSON Schema
<code>Field(description=...)</code>	→ <code>"City to look up."</code>	what goes in the argument

DESIGN RULE

Write tools the way you would write a public API for a junior engineer with no context: narrow scope, unambiguous name, explicit units, errors returned as readable strings the model can act on.

THE DOCSTRING IS AN API

It is not a comment for humans. It is the only thing telling the model when this tool is the right choice.

Vague docstring → the model guesses. Guessing looks like a flaky agent, and you will debug the prompt for a day before you look here.

The tool-call loop



- 1 Model returns a tool call**
name + JSON arguments, not prose
- 2 Framework validates & dispatches**
schema check, then your Python runs
- 3 Result goes back as a message**
the model now has new facts
- 4 Model decides again**
answer, or call another tool — repeat

BOUND THE LOOP

Nothing stops a model calling a tool forever except limits you set: max iterations, token budget, timeouts, and idempotent tools.

Tools in practice

```
from agent_framework import Agent, tool
from pydantic import Field

@tool(approval_mode="never_require")
def get_weather(
    location: Annotated[str, Field(description="The location.")],
) -> str:
    """Get the weather for a given location."""
    conditions = ["sunny", "cloudy", "rainy", "stormy"]
    return f"{location} is {choice(conditions)}, high {randint(10, 30)}C"

agent = Agent(
    client=OpenAIChatCompletionClient(),
    name="WeatherAgent",
    instructions="Use get_weather to answer.",
    tools=[get_weather],
)
```

- **tools= takes a list**
Or a single tool — both accepted.
- **Plain functions work too**
The decorator adds approval control.
- **Instructions steer usage**
Name the tool if the model hesitates.
- **Async tools supported**
Use async def for I/O-bound work.

Demo — tools and the call loop

[▶ DEMO](#)**terminal**

```
uv run python examples/02_add_tools.py

# then try, live:
# - ask about two cities in one message
# - ask something with no tool at all
# - break the docstring and watch selection degrade
```

WHAT TO POINT AT

- ▶ The model emits a tool call, not prose — show the arguments.
- ▶ Two cities in one prompt can produce two tool calls.
- ▶ Off-topic questions should NOT trigger the tool.
- ▶ Vandalise the docstring to "gets data" and re-run.

Memory & context

Three horizons: the turn, the session, and everything before

sessions

providers

persistence

compaction

04

Three horizons of memory

The turn

`message list`

What you pass into a single run. Always present, never persisted.

The session

`AgentSession`

Conversation state across runs. In-memory by default; pluggable stores.

Beyond the session

`ContextProvider + store`

Facts, preferences, prior conversations — retrieved and injected per run.

grows →

```
session = agent.create_session()           # one conversation
await agent.run("My name is Alice.", session=session)
await agent.run("What do you remember?", session=session) # knows
```

Sessions: multi-turn in three lines

```
agent = Agent(  
    client=OpenAIChatCompletionClient(),  
    name="ConversationAgent",  
    instructions="Keep your answers brief.",  
)  
  
session = agent.create_session()  
  
await agent.run("My name is Alice and I love hiking.", session=session)  
  
# same session → the agent still has the earlier turns  
result = await agent.run("What do you remember about me?",  
                          session=session)
```

- **Omit session → amnesia**

Each run starts clean. Sometimes that's what you want.

- **One session per conversation**

Never share one across users or requests.

- **session.state is a dict**

Providers namespace their data by source_id.

- **Swap the backing store**

InMemory, Redis, Cosmos — same API.

Context providers: hooks around every run

examples/04_memory.py

```
class UserMemoryProvider(ContextProvider):
    async def before_run(self, *, agent, session, context, state):
        if name := state.get("user_name"):
            context.extend_instructions(
                self.source_id, f"The user's name is {name}."

    async def after_run(self, *, agent, session, context, state):
        for msg in context.input_messages:
            if "my name is" in (msg.text or "").lower():
                state["user_name"] = extract(msg.text)

agent = Agent(..., context_providers=[UserMemoryProvider()])
```

before_run

read state → inject context

after_run

observe the turn → write state

state is namespaced per provider by `source_id`, so two providers never collide — and everything they write rides along in the session.

THIS IS THE RAG SEAM

Retrieval is just a `before_run` that queries your store and calls `extend_instructions`. There is no separate 'RAG mode' — memory, retrieval and personalisation are all the same extension point.

Providers stack

Same two hooks, different jobs. Order in the list is the order they run.

InMemoryHistoryProvider

the transcript

load_messages=True — replays history into every run

RedisContextProvider

semantic memory

durable facts retrieved across sessions

InMemoryHistoryProvider

the audit copy

load_messages=False — records, never feeds the model

```
context_providers=[
    transcript,
    agent_memory,
    audit,    # last
]
```

The audit store goes last so
store_context_messages=True captures what the
providers above it injected.

EACH PROVIDER OWNS A SLICE OF SESSION.STATE

Namespaced by source_id, so two providers never collide. In the example the transcript holds 4 messages and the audit copy 6 — same run, different jobs.

Persistence: outliving the process

The provider persists

`FileHistoryProvider(path)`

Append-only file per session. Reuse the session_id and the previous run's history is already there.

Redis and Cosmos providers, same shape.

You persist

`session.to_dict()`

The session is just a dict. Serialise it into whatever store you already run, then `AgentSession.from_dict()` to resume.

```
blob = json.dumps(session.to_dict())          # ~530 bytes
restored = AgentSession.from_dict(json.loads(blob))
await new_agent.run("What is my favourite colour?", session=restored)
```

THE PRODUCTION QUESTION THIS ANSWERS

"Where do sessions live when the process restarts?" — pick one of these two before you ship, not after.

Compaction: choosing what to forget

Long conversations do not fail gracefully. They fail at token limit $N+1$ — mid-sentence, in production.

SlidingWindow

keep the last N groups

Truncation

over N messages? trim to M

ToolResultCompaction

summarise old tool groups, keep the trace

SelectiveToolCall

drop old tool groups outright

ContextWindow

derive the budget from the model

Summarization

fold old turns into prose (needs a model)

TokenBudgetComposedStrategy — run several, cheapest first, until the budget is met

THEY ARE NOT INTERCHANGEABLE

Dropping a tool group reclaims more tokens than summarising it — and loses the trace. Summarising costs a model call. Example 08 runs all seven side by side so you can see the trade in numbers.

Demo — sessions, providers, persistence, compaction

 DEMO**terminal**

```
uv run python examples/03_multi_turn.py
uv run python examples/05_memory_providers.py
uv run python examples/06_persistent_history.py    # run it twice

uv run python examples/08_compaction_strategies.py
```

WHAT TO POINT AT

- ▶ 03: drop session= on turn two — the agent forgets Alice.
- ▶ 05: transcript holds 4 messages, the audit copy 6.
- ▶ 06: run twice — the on-disk record count grows 2 → 4.
- ▶ 08: before/after token counts for every strategy.

IF THE DEMO GODS SAY NO

08 needs no model for six of its seven strategies, and 11/12/13/14 need none at all — plenty to show if the model server stalls.

MCP

Stop writing one-off integrations for every tool and every agent

the N×M problem

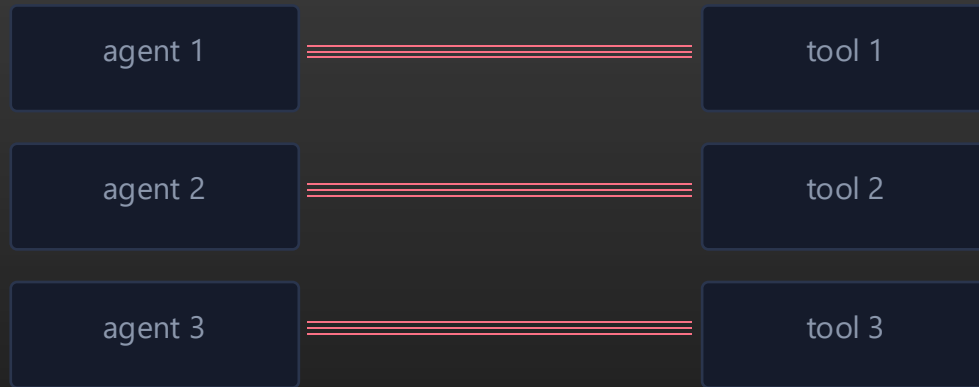
transports

trust

05

Model Context Protocol: $N \times M \rightarrow N + M$

WITHOUT A PROTOCOL



9 bespoke integrations · 9 places to break

WITH MCP



6 connections · one contract to test

WHY THIS MATTERS BEYOND CONVENIENCE

An MCP server is a deployable, versioned, independently-owned unit. The team that owns the CRM ships the CRM tool server; your agent consumes it without a code change on either side.

It also moves the trust boundary somewhere you can actually govern — authn, rate limits and audit live at the server, not scattered through agent code.

Connecting a tool server

```
from agent_framework import Agent, MCPStreamableHTTPTool

async with Agent(
    client=OpenAIChatCompletionClient(),
    name="DocsAgent",
    instructions="You help with Microsoft documentation questions.",
    tools=MCPStreamableHTTPTool(
        name="Microsoft Learn MCP",
        url="https://learn.microsoft.com/api/mcp",
    ),
) as agent:
    async for chunk in agent.run(query, stream=True):
        ...
```

- **async with is required**

The tool owns a connection; the agent owns its lifetime.

- **Tools are discovered**

The server advertises its catalogue at connect time.

- **One line, many tools**

A server can expose dozens — you wire up none of them.

TRANSPORTS

Streamable HTTP for remote servers · stdio for local processes

Demo — an agent with a documentation tool server

▶ DEMO

terminal

```
uv run python examples/09_mcp_docs_agent.py

# question it answers:
# "How do I create an Azure storage account with the az CLI?"
```

WHAT TO POINT AT

- ▶ No tool schemas anywhere in the file — they came from the server.
- ▶ The answer contains real commands, not model recall.
- ▶ Ask a follow-up about a service the model predates.
- ▶ Note the latency: remote tool + streaming generation.

IF THE DEMO GODS SAY NO

Requires outbound internet to learn.microsoft.com. Have a screenshot ready, and fall back to walking the nine lines of code.

Workflows

When one agent in a loop is the wrong shape for the problem

functional

graph

executors

edges

orchestration

06

Autonomy versus control

Agent decides

The model chooses the next step. Flexible, handles the unexpected, hard to predict or test.

Good for: open-ended research, support triage, anything where the path is not known in advance.

You decide

The topology is code. Deterministic, testable, observable — and it will not improvise past what you wrote.

Good for: pipelines, approvals, anything with an SLA or an auditor.

Most real systems are both: a deterministic workflow whose nodes happen to be agents.

```
@workflow                                # functional – plain async Python, easy to read
WorkflowBuilder(...)                     # graph – executors + edges, inspectable topology
```

Functional workflows

```
from agent_framework import Agent, workflow

writer = Agent(name="WriterAgent", client=client,
               instructions="Write a short poem about the topic.")
reviewer = Agent(name="ReviewerAgent", client=client,
                 instructions="Review the poem in one sentence.")

@workflow
async def poem_workflow(topic: str) -> str:
    poem = (await writer.run(f"Write about: {topic}")).text
    review = (await reviewer.run(f"Review: {poem}")).text
    return f"{poem}\n\nReview: {review}"

result = await poem_workflow.run("a cat learning to code")
print(result.get_outputs()[0])
```

- **It reads like Python**

Because it is. Control flow is if/for/await.

- **Agents are just callables**

Compose them however you compose functions.

- **Outputs are a collection**

get_outputs() — a run can yield more than one.

- **Best first choice**

Reach for graphs when you need topology, not before.

Graph workflows

```
class UpperCase(Executor):
    @handler
    async def to_upper(self, text: str, ctx: WorkflowContext[str]):
        await ctx.send_message(text.upper())

    @executor(id="reverse_text")
    async def reverse_text(text: str, ctx: WorkflowContext[Never, str]):
        await ctx.yield_output(text[::-1])

upper = UpperCase(id="upper_case")
wf = (WorkflowBuilder(start_executor=upper)
      .add_edge(upper, reverse_text)
      .build())

events = await wf.run("hello world")  # ['DLROW OLLEH']
```

upper_case

Executor + @handler

**reverse_text**

@executor → yield_output

Nodes are executors, edges route messages.
send_message passes on; yield_output ends the run
with a result.

runs with no model at all

Executors: Two ways to declare a node

```
class Ingest(Executor):                                # 1. class-based
    @handler
    async def from_text(self, t: str, ctx: WorkflowContext[str]):
        await ctx.send_message(t)

    @executor(id="normalise")                           # 2 function-based
    async def normalise(t: str, ctx: WorkflowContext[str]):
        await ctx.send_message(t.lower())
```

THE CONTEXT TYPES ARE THE CONTRACT

WorkflowContext

side effects only

WorkflowContext[T]

sends T downstream

WorkflowContext[Never, U]

yields U, sends nothing

WorkflowContext[T, U]

sends T and yields U

Validated at build time — a mismatch fails the build, not the run.

Edges: six ways to wire the graph

add_chain([a, b, c])

linear pipeline — the 90% case

add_edge(a, b, condition=...)

take the edge only if a predicate holds

add_fan_out_edges(a, [b, c])

same message to several nodes, in parallel

add_fan_in_edges([b, c], d)

d receives a list of collected results

add_switch_case_edge_group(...)

Case / Default — exactly one branch wins

add_multi_selection_edge_group(...)

pick a subset of targets at runtime

```
wf = (WorkflowBuilder(
    start_executor=trriage)
    .add_switch_case_edge_group(
        triage, [
            Case(condition=is_sev1,
                 target=pager),
            Default(target=backlog),
        ])
    .build())
```

TWO GOTCHAS

Conditional edges are evaluated independently
— overlapping predicates run both branches.
A switch group needs a Default.

Execution: supersteps, events and shared state

Workflows advance in supersteps — each one delivers a round of messages, which is why parallel branches appear grouped.

workflow.run(x, stream=True)

```
-- superstep 1
  executor_invoked    parse    laptop, dock
  intermediate        parse    parsed 2 items
  executor_completed  parse    [['laptop', 'dock'], ...]
-- superstep 2
  executor_invoked    price    ['laptop', 'dock']
  intermediate        price    priced 2 items
-- superstep 3
  executor_invoked    invoice  {'laptop': 60, ...}
  output              invoice  invoice for 2 items ...
```

ctx.set_state / get_state

scratch space every executor in the run can read

get_outputs()

the terminal answer — from output_from nodes

get_intermediate_outputs()

progress — from intermediate_output_from nodes

STATE IS PER RUN, NOT PER WORKFLOW

Two runs of the same workflow object do not see each other's state — so a built workflow is safe to reuse across requests.

Orchestration patterns

Sequential

`SequentialBuilder`

A → B → C. Pipelines, review chains.

Concurrent

`ConcurrentBuilder`

Fan out, gather, aggregate.

Handoff

`HandoffBuilder`

Route to the specialist, transfer control.

Group chat

`GroupChatBuilder`

Shared transcript, managed turn-taking.

Magentic

`MagenticBuilder`

A manager plans and delegates dynamically.

PICK THE DULLEST

Cost and unpredictability climb as you go down this list.

SEQUENCING ADVICE

Start sequential. Move to concurrent when latency hurts. Reach for magentic only when you genuinely cannot enumerate the steps — it is the most capable and the least predictable option here.

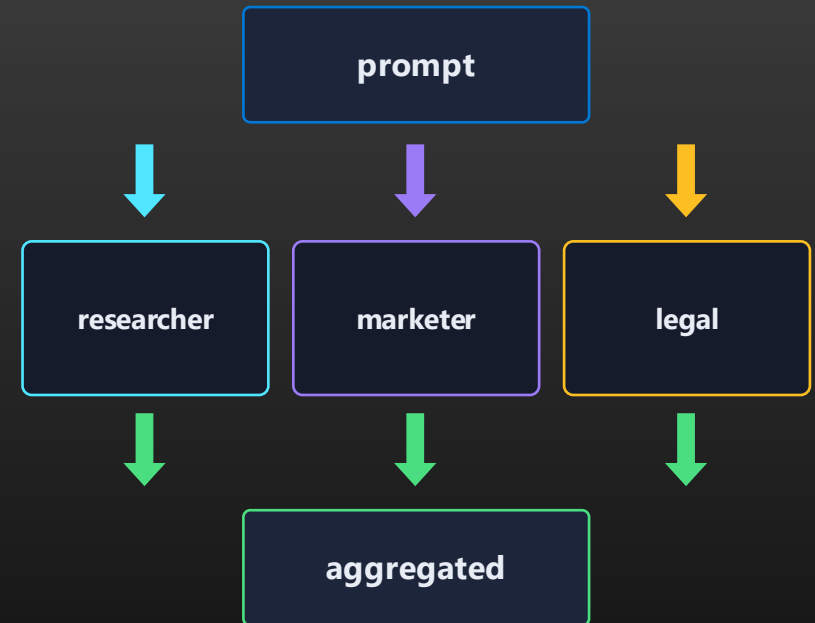
Fan-out, fan-in

```
from agent_framework.orchestrations import ConcurrentBuilder

researcher = Agent(client=client, name="researcher",
                    instructions="Concise, factual market insights.",
                    default_options={"max_tokens": 500})
marketer = Agent(..., name="marketer")
legal = Agent(..., name="legal")

workflow = ConcurrentBuilder(
    participants=[researcher, marketer, legal]).build()

events = await workflow.run("We are launching a budget e-bike.")
for msg in events.get_outputs()[0].messages:
    print(msg.author_name, msg.text)
```



Wall clock = the slowest participant.
author_name tells you who said what.

Demo — workflows, end to end

▶ DEMO

terminal

```
# these four need no model at all
uv run python examples/11_graph_workflow.py
uv run python examples/12_executor_types.py
uv run python examples/13_edge_patterns.py
uv run python examples/14_workflow_execution.py

uv run python examples/10_functional_workflow.py
uv run python examples/15_concurrent_orchestration.py
```

WHAT TO POINT AT

- ▶ 12: one node, two input types — dispatch is on type.
- ▶ 13: six routing patterns, each printing its decision.
- ▶ 14: the superstep timeline, live.
- ▶ 10/15: agents as nodes; 15 prints author_name.

IF THE DEMO GODS SAY NO

Four of these are pure Python. If the model server is struggling, run 11, 12, 13 and 14, then read 10 and 15 as code with the diagrams on screen.

Middleware

Cross-cutting concerns that do not belong in your agent code

agent vs function

three flavours

short-circuit

07

Two pipelines, two scopes

AGENT MIDDLEWARE · once per run

FUNCTION MIDDLEWARE · once per tool call

model call
+ tool dispatch

One agent middleware wraps the entire exchange — including every tool round trip inside it.
One function middleware fires per individual tool invocation.

Guardrails

block unsafe input before it reaches the model

Auth & tenancy

attach identity, enforce scope per tool

Telemetry

spans, timings, token accounting

Caching & retries

short-circuit repeats, wrap flaky tools

Three ways to write it

Class-based

`AgentMiddleware`

`FunctionMiddleware`

Stateful, testable, injectable. Reach for it when the middleware has dependencies or configuration.

[examples/12](#)

Function-based

```
async def mw(context, call_next)
```

Detected from the context type hint.
Lightest option for stateless concerns.

[examples/13](#)

Decorator

`@agent_middleware`

`@function_middleware`

Explicit intent, no type annotations needed. Mismatches are caught at registration.

[examples/14](#)

```
agent = Agent(..., middleware=[SecurityAgentMiddleware(), LoggingFunctionMiddleware()])  
# both pipelines registered in one list – the framework sorts them by type
```

Short-circuiting: a guardrail in twelve lines

```
class SecurityAgentMiddleware(AgentMiddleware):
    async def process(self, context, call_next):
        last = context.messages[-1] if context.messages else None
        if last and last.text and risky(last.text):
            context.result = AgentResponse(messages=[
                Message("assistant", ["Request blocked."])]
            )
            return # never call call_next → model never runs
        await call_next()

class LoggingFunctionMiddleware(FunctionMiddleware):
    async def process(self, context, call_next):
        start = time.time()
        await call_next()
        log(context.function.name, time.time() - start)
```

- **Return without call_next**

The model is never invoked. No tokens spent.

- **Set context.result**

Otherwise the caller gets an empty response.

- **Time around call_next**

Before/after is where telemetry lives.

- **Order is the list order**

First registered is outermost.

Demo — guardrails and timing

▶ DEMO

terminal

```
uv run python examples/16_class_based_middleware.py

# same behaviour, different ergonomics:
uv run python examples/18_decorator_middleware.py
```

WHAT TO POINT AT

- ▶ Normal query: security passes, tool runs, timing logged.
- ▶ "What's the password...": blocked before the model is called.
- ▶ Note the log ordering — outer in, inner in, inner out, outer out.
- ▶ 18 shows the decorator form with untyped parameters.

IF THE DEMO GODS SAY NO

Example 17 uses FoundryChatClient and needs `az login` — use 16 and 18, which run against the local model.

A2A Protocol

Agent to Agent Protocol

Remote Agent

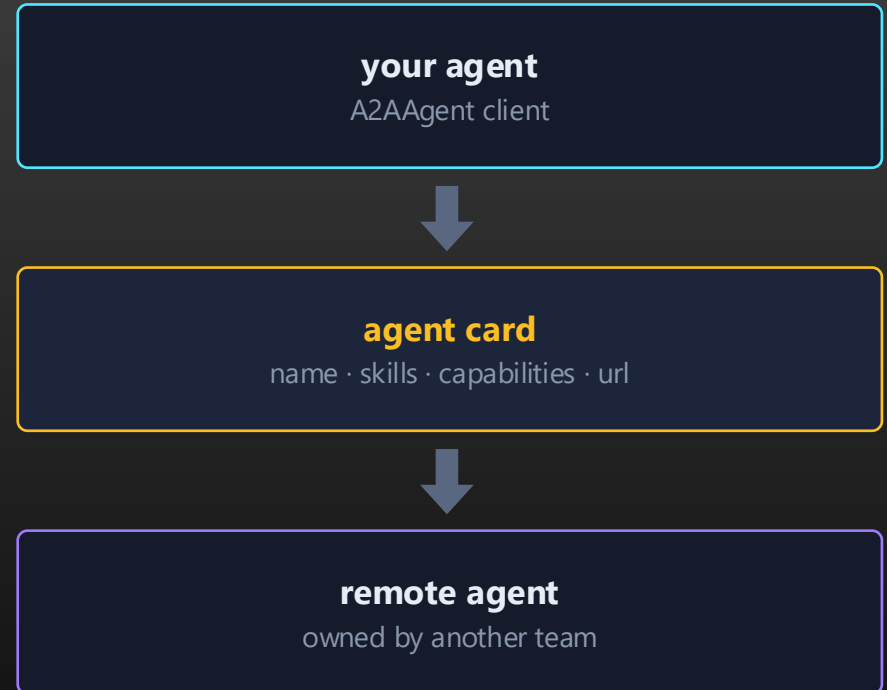
A2A

08

A2A: agents as network citizens

```
# server – publish a card describing what this agent can do
card = AgentCard(name="Europe Travel Agent", version="1.0.0",
                 skills=[flight_skill],
                 capabilities=AgentCapabilities(streaming=True))
handler = DefaultRequestHandler(
    agent_executor=A2AExecutor(agent, stream=True),
    task_store=InMemoryTaskStore(), agent_card=card)

# client – discover, then call
card = await A2ACardResolver(...).get_agent_card()
async with A2AAgent(name=card.name, agent_card=card,
                   url=host) as remote:
    response = await remote.run("search flights for Europe")
```



Discovery, not hard-coded endpoints — the card is the contract.

Running against a local model

```
# vllm-run.sh
vllm serve Qwen/Qwen3.5-4B \
  --max-model-len=8192 \
  --enable-prefix-caching \
  --enable-auto-tool-choice \
  --tool-call-parser qwen3_coder

# .env
OPENAI_API_KEY=test-key           # placeholder, unused
OPENAI_MODEL=Qwen/Qwen3.5-4B
OPENAI_BASE_URL=http://localhost:8000/v1
```

- **Tool calling needs flags**

--enable-auto-tool-choice and a matching parser.

- **8k window is the constraint**

Hence max_tokens everywhere in the examples.

- **Chat Completions only**

OpenAIChatCompletionClient, not OpenAIChatClient.

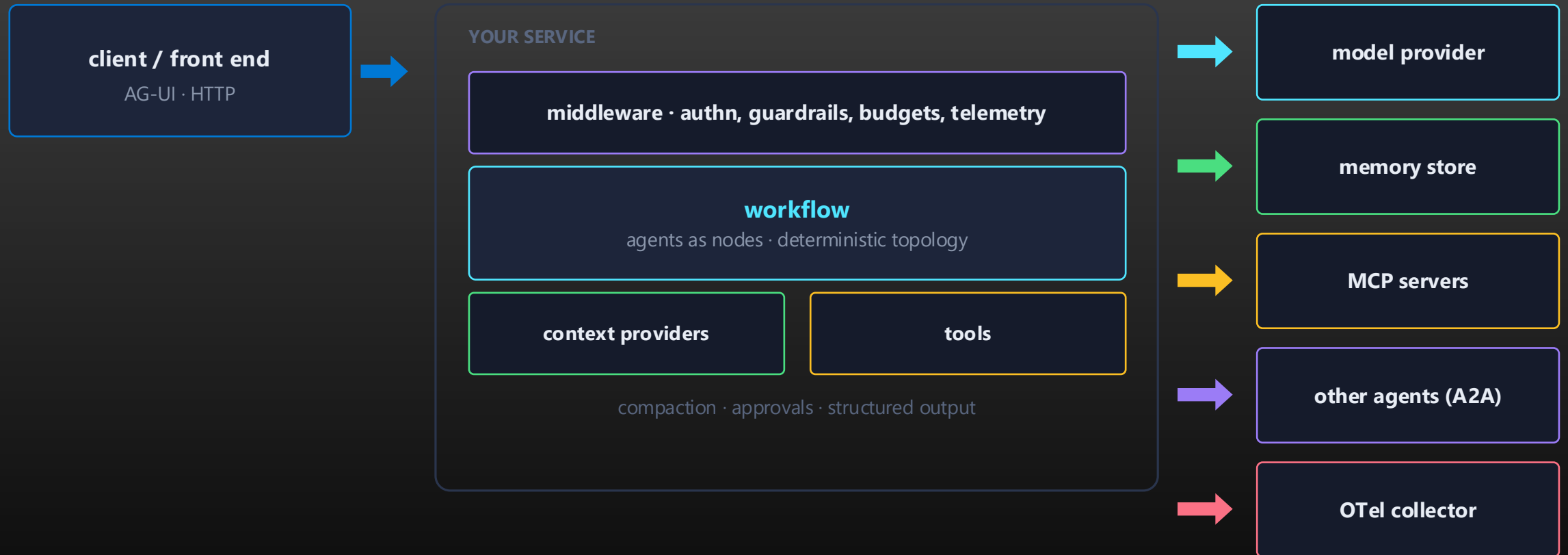
- **Same code, any backend**

Point OPENAI_BASE_URL elsewhere and nothing changes.

WHY BOTHER WITH A LOCAL MODEL

Free iteration, no rate limits, no data leaving the machine — and a small model is an honest test. Prompts and tool descriptions that only work on a frontier model are prompts that will surprise you later.

Putting it together



Deterministic skeleton, agentic joints, governed edges.

The lab, end to end

01	hello_agent	agent, streaming	11	graph_workflow	executors + edges
02	add_tools	typed tools	12	executor_types	4 executor forms
03	multi_turn	sessions	13	edge_patterns	6 wiring patterns
04	memory	ContextProvider	14	workflow_execution	supersteps, events
05	memory_providers	layered providers	15	concurrent_orchestration	fan-out / fan-in
06	persistent_history	file + to_dict	16	class_basedmiddleware	guardrail + timing
07	context_compaction	CompactionProvider	17	function_basedmiddleware	async fn (Foundry)
08	compaction_strategies	all 7 strategies	18	decoratormiddleware	@agentmiddleware
09	mcp_docs_agent	MCP over HTTP	19	response_format	structured output
10	functional_workflow	@workflow	20	harness_agent	planning loop

plus dev_ui/ · agent_ui/ · others/ for DevUI, AG-UI, A2A and Azure Functions hosting

Now you build it

Eight self-checking tasks. Each one fails with a hint until you finish it — five need no model.

01 tool_design

docstrings and Field descriptions as the model's API

02 context_provider

before_run / after_run and provider state

03 compaction_budget

composing strategies under a token budget

04 executor_contract

WorkflowContext type parameters

05 routing

Case / Default switch-case routing

06 fan_in

what a fan-in target actually receives

07 guardrail_middleware

short-circuiting before the model runs

08 structured_output

response_format and the failure branch

```
uv run python exercises/01_tool_design.py      # read the FAILs, edit, repeat
uv run python exercises/run_all.py --offline    # the five that need no model
```

Worked answers in exercises/solutions/ — read them once you have something failing for the right reason.

Where to go next

Documentation

learn.microsoft.com/agent-framework

Concepts, API reference, migration guidance from AutoGen and SK.

Source & samples

github.com/microsoft/agent-framework

python/samples is far broader than this lab — read it.

Model Context Protocol

modelcontextprotocol.io

The spec, plus a directory of existing servers.

This workshop

<https://github.com/satendrakumar/learning-ms-agent-framework>

Setup, the ordered lab, and stretch goals.

BEFORE YOU LEAVE

Work through exercises/ — eight self-checking tasks, five of which need no model. Then: rebuild example 10 with SequentialBuilder, or set approval_mode="always_require" on a tool and watch the flow.

Questions

Deterministic skeleton. Agentic joints. Governed edges.

The framework is not the interesting part — what you can now safely put in production is.

`uv sync`

`./vllm-run.sh`

`uv run python examples/01_hello_agent.py`

Satendra Kumar

Everything in this deck is in the repo — examples/README.md is the lab guide.